LAPORAN PRAKTIKUM

POSTTEST 1

ALGORITMA PEMROGRAMAN LANJUT

Disusun oleh:

Muhamad Radja Nur Akbar (2509106012)

Kelas (A1'25)

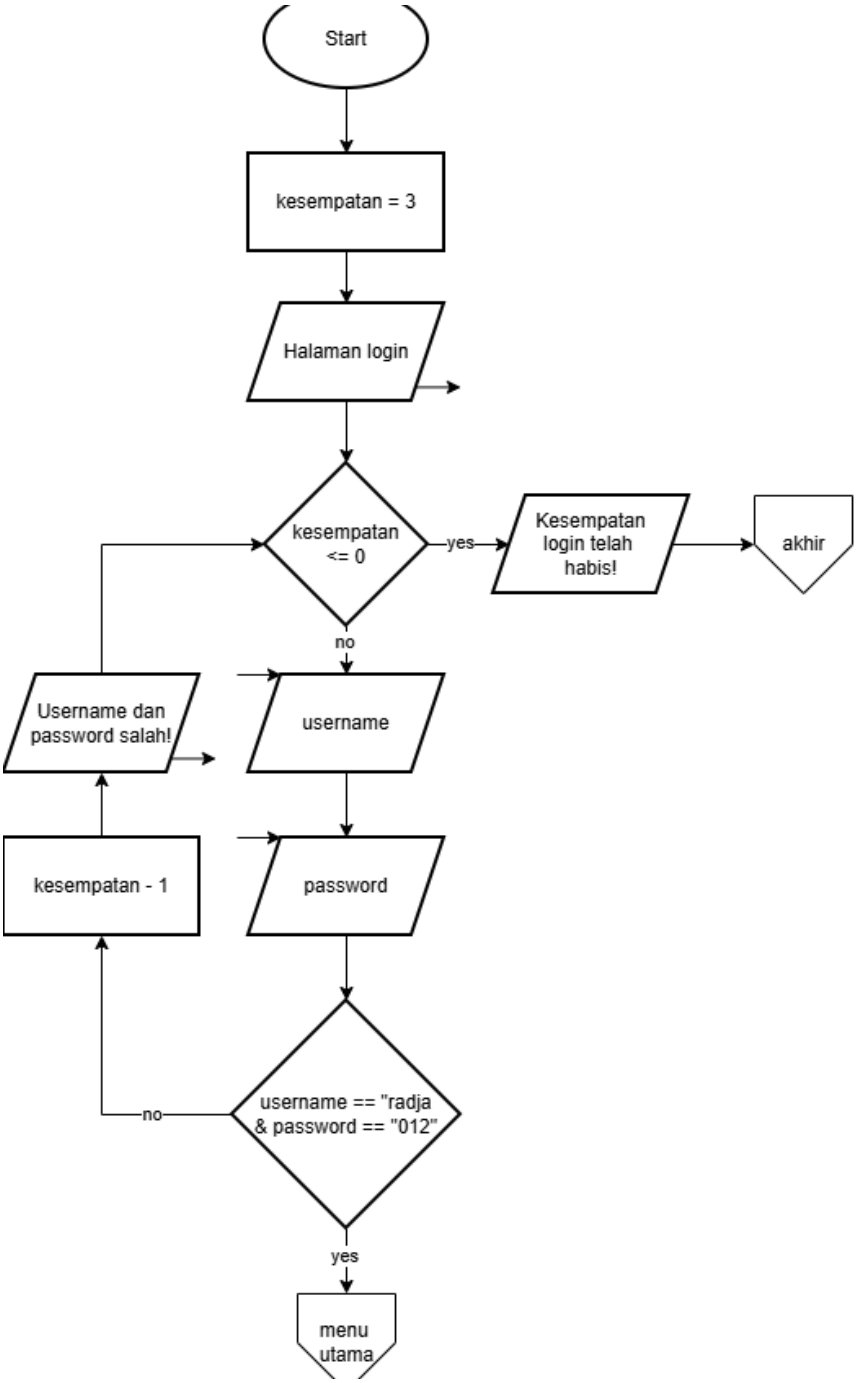
PROGRAM STUDI INFORMATIKA

UNIVERSITAS MULAWARMAN

SAMARINDA

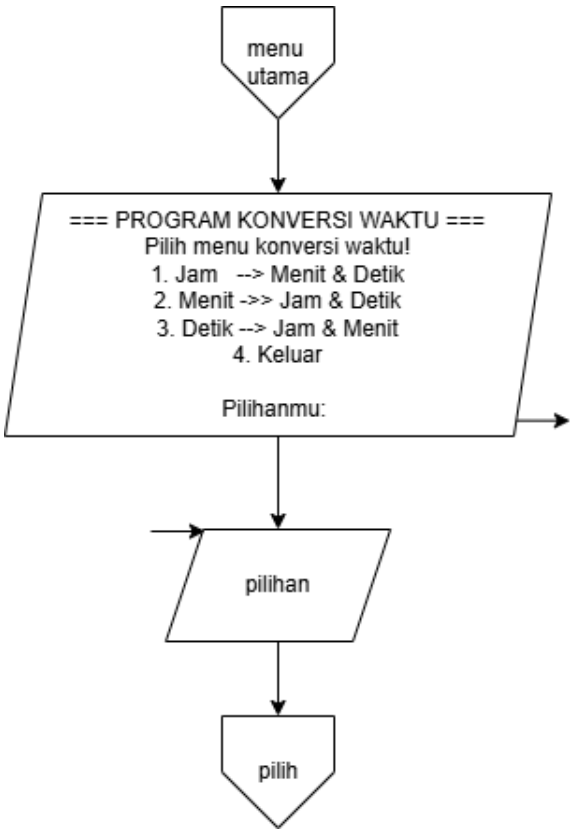
2026

1. Flowchart



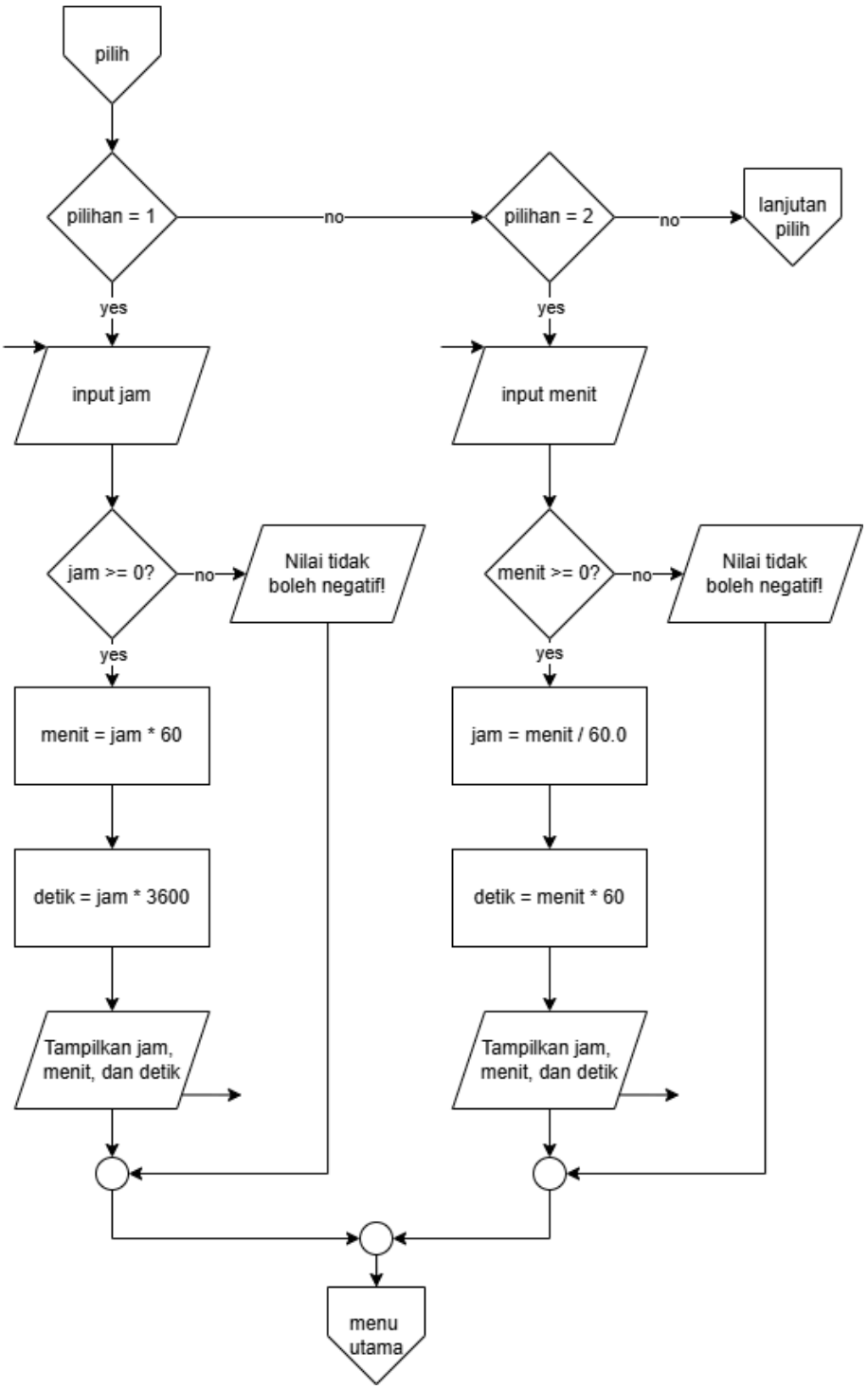
Gambar 1.1 Halaman Login

Sebelum memulai program pengguna diminta untuk *login* terlebih dahulu. Setiap pengguna memiliki tiga kesempatan *login*, setiap *username* dan *password* yang salah akan mengurangi kesempatan sebanyak satu dan pengguna harus input ulang. Ketika kesempatan habis program akan berakhir. Ketika *login* berhasil pengguna masuk ke menu utama.



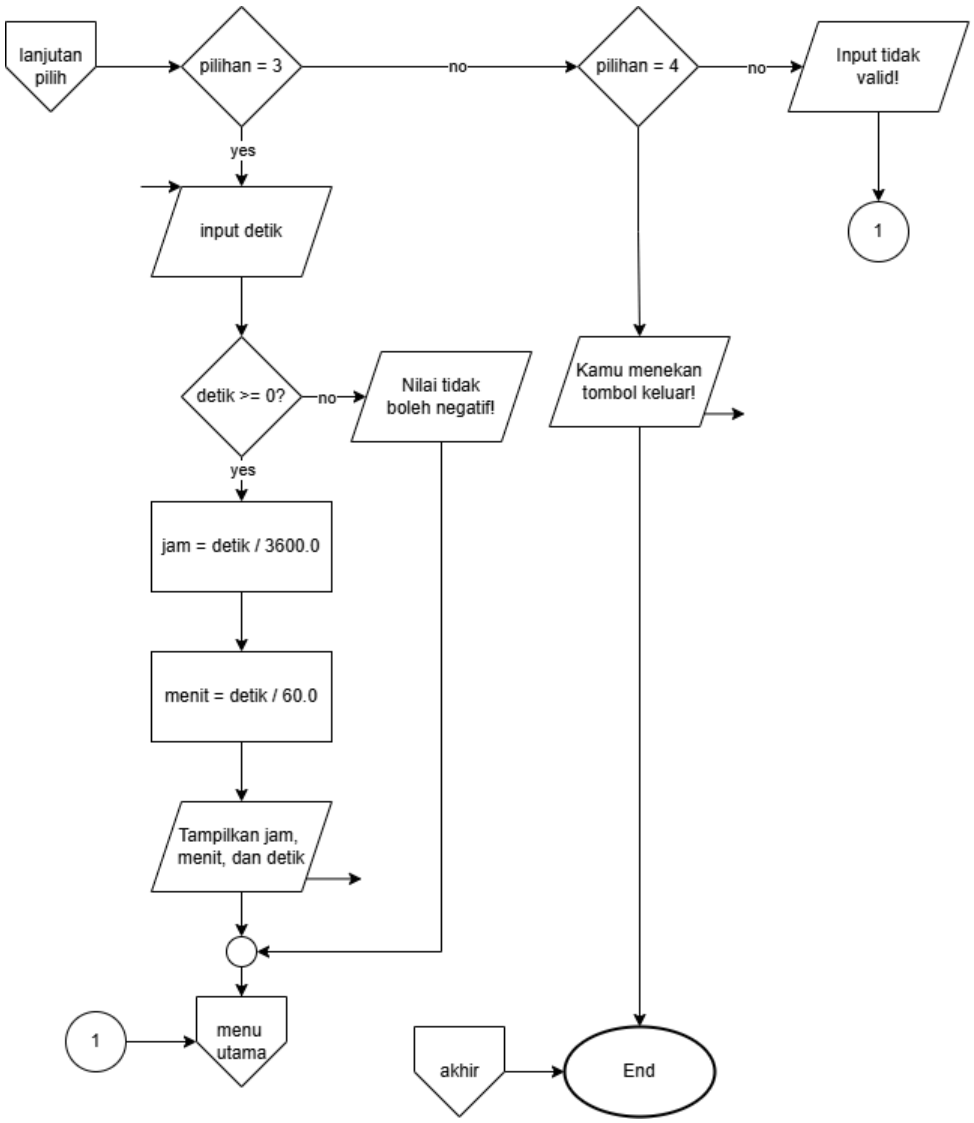
Gambar 1.2 Menu Utama

Dalam program konversi waktu ini, pengguna bisa memilih satuan waktu apa yang akan dikonversi seperti pada gambar. Memilih pilihan keempat akan langsung mengakhiri program.



Gambar 1.3 Pilihan Konversi

Dari ketiga menu konversi semua menjalankan prosedur yang sama. Meminta input pengguna berupa nilai yang lebih atau sama dengan nol. Kemudian memulai konversi waktu antara detik, menit, dan jam yang ditunjukkan seperti pada potongan *flowchart* di atas. Hasil ditampilkan dan pengguna kembali ke menu utama.



Gambar 1.4 Lanjutan Menu Konversi

Pengguna juga dibawa kembali ke menu utama jika memasukkan pilihan yang tidak sesuai.

2. Deskripsi Singkat Program

Satuan waktu adalah standar ukuran durasi, dengan detik (*s*) sebagai satuan pokok. Selain detik terdapat pula satuan menit, jam, hari, dll. Program ini memungkinkan kalian untuk mengonversi/mengubah satu satuan waktu ke satuan lainnya dibatasi di detik, menit, dan jam sesuai rumus dan ketentuan yang ada.

3. Source Code

```
#include <iostream>
using namespace std;

int main() {
    double detik, menit, jam;
    int pilihan;
    int kesempatan = 3;
    string usr = "radja", pw = "012", username, password;
```

Gambar 3.1 Inisialisasi dan Deklarasi

Langkah pertama adalah menginisialisasi beberapa variabel yang akan digunakan. Penggunaan tipe data *double* pada tiga satuan waktu utama sangatlah penting untuk meningkatkan akurasi dan presisi konversi jika ada bilangan desimal yang banyak. Lalu variabel 'kesempatan' sebagai berapa kali pengguna bisa *login*. Adapun *username* dan *password* yang sudah ditetapkan dengan tipe data *string*.

```
    cout << "Selamat datang di Time Flies! Konversi satuan waktu ke satuan lain!\n\n";
    while (kesempatan > 0) {
        cout << "Silahkan login untuk melanjutkan\n";
        cout << "Username: ";
        cin >> username;
        cout << "Password: ";
        cin >> password;
```

Gambar 3.2 Kode Tampilan dan Input Login

Selama 'kesempatan' lebih dari 0 (sebelumnya ditetapkan 3) pengguna bisa terus memasukkan akun yang benar jika ada kesalahan.

```
        if (username == usr && password == pw) {
            cout << "Login berhasil!\n";
            break;
        } else {
            kesempatan--;
            cout << "Username dan password salah!\n\n";
            cout << "Sisa kesempatan login: " << kesempatan << endl;
        }
    } if (kesempatan <= 0) {
        cout << "Kesempatan login telah habis, coba lagi nanti!\n";
        return 0;
    }
```

Gambar 3.3 Kondisi Login Sukses

Variabel ‘usr’ dan ‘pw’ merupakan variabel yang menyimpan nilai yang sudah tetap mengingat hanya ada satu akun, sementara variabel ‘username’ dan ‘password’ adalah variabel untuk menyimpan input pengguna. Jika berhasil pengguna langsung lanjut ke menu konversi. Jika salah maka harus input ulang dan jika kesempatan habis maka program langsung dihentikan.

```
    while(true) {
        cout << "=== PROGRAM KONVERSI WAKTU ===\n\n";
        cout << "Pilih menu konversi waktu!\n";
        cout << "1. Jam     --> Menit & Detik\n";
        cout << "2. Menit   --> Jam & Detik\n";
        cout << "3. Detik   --> Jam & Menit\n";
        cout << "4. Keluar\n";
        cout << "Pilihanmu: ";
        if (!(cin >> pilihan)) {
            cin.clear();
            cin.ignore(100, '\n');
            cout << "Input tidak valid!\n\n";
            continue;
        }
```

Gambar 3.4 Tampilan Menu Konversi

Pengguna bisa memilih tiga konversi utama seperti pada gambar. Ketika pengguna memasukkan input selain angka maka program langsung meminta pengguna input ulang dan notifikasi kesalahan input. Ketika ‘cin’ gagal membaca ia akan beku dan ‘cin.clear()’ yang mengembalikannya ke semula. Adapun ‘cin.ignore()’ akan membuat sisa karakter pada *buffer*, maks 100 karakter atau saat membaca *enter/newline.* ‘Continue’ melewati sisa kode di *switch* dan kembali ke menu konversi lagi.

```
45  switch(pilihan) {
46      case 1: {
47          cout << "Masukkan jam: ";
48          cin >> jam;
49          if (jam < 0) {
50              cout << "Nilai tidak boleh negatif!\n\n";
51              break;
52          }
53          menit = jam * 60;
54          detik = jam * 3600;
55          cout << jam << " " << "Jam = " << menit << " " << "Menit = " << detik << " " << "Detik\n\n";
56          break;
57      }
```

Gambar 3.5 Konversi Jam ke Menit & Detik

Menu pertama berisi konversi satuan jam ke menit dan detik. Pertama-tama pengguna harus menginput nilai jam dan tidak boleh bilangan negatif, jika masih dipaksa akan kembali ke menu konversi. Sesuai rumus konversi yang ada pengonversian dimulai, sesudah itu hasil ditampilkan dan kembali ke menu konversi.

```
58      case 2: {
59          cout << "Masukkan menit: ";
60          cin >> menit;
61          if (menit < 0) {
62              cout << "Nilai tidak boleh negatif!\n\n";
63              break;
64          }
65          jam = menit / 60.0;
66          detik = menit * 60;
```

Gambar 3.6 Konversi Menit ke Jam & Detik

```
70      case 3: {
71          cout << "Masukkan detik: ";
72          cin >> detik;
73          if (detik < 0) {
74              cout << "Nilai tidak boleh negatif!\n\n";
75              break;
76          }
77          jam = detik / 3600.0;
78          menit = detik / 60.0;
```

Gambar 3.7 Konversi Detik ke Jam dan Menit

Proses yang sama juga terjadi di pilihan lainnya (*case* 2 dan *case* 3), perbedaan hanya pada rumus mengikuti rumus yang sesuai.

```
82          case 4: {
83              cout << "Kamu menekan tombol keluar!\n";
84              return 0;
85          }
86          default: {
87              cout << "Pilihan tidak valid!\n\n";
88              break;
89          }
```

Gambar 3.8 Case Lain

Seperti pada menu memilih pilihan keempat akan mengakhiri program. *Case default* berperan sebagai blok kode yang mengeksekusi nilai dari *switch* yang tidak cocok dengan *case* yang sudah ada. Blok kode ini akan jalan jika pengguna menginput angka selain keempat *case* yang ada.

4. Hasil Output

```
Silahkan login untuk melanjutkan
Username: rad
Password: 011
Username dan password salah!

Sisa kesempatan login: 2
Silahkan login untuk melanjutkan
Username: kuda
Password: 098
Username dan password salah!

Sisa kesempatan login: 1
Silahkan login untuk melanjutkan
Username: fr
Password: 98
Username dan password salah!

Sisa kesempatan login: 0
Kesempatan login telah habis, coba lagi nanti!
```

Gambar 4.1 Kesempatan Login Habis

```
=== PROGRAM KONVERSI WAKTU ===

Pilih menu konversi waktu!
1. Jam      --> Menit & Detik
2. Menit    --> Jam & Detik
3. Detik    --> Jam & Menit
4. Keluar
```

Gambar 4.2 Tampilan Menu Konversi

```
Pilihanmu: 1
Masukkan jam: 2
2 Jam = 120 Menit = 7200 Detik
```

Gambar 4.3 Pilihan Jam ke Menit & Detik

```
Pilihanmu: 2
Masukkan menit: 240
240 Menit = 4 Jam = 14400 Detik
```

Gambar 4.4 Pilihan Menit ke Jam & Detik

```
Pilihanmu: 3
Masukkan detik: 10000
10000 Detik = 2.77778 Jam = 166.667 Menit
```

Gambar 4.5 Pilihan Detik ke Jam & Menit

```
Pilih menu konversi waktu!
1. Jam      --> Menit & Detik
2. Menit    --> Jam & Detik
3. Detik    --> Jam & Menit
4. Keluar
Pilihanmu: 5
Pilihan tidak valid!

=== PROGRAM KONVERSI WAKTU ===

Pilih menu konversi waktu!
1. Jam      --> Menit & Detik
2. Menit    --> Jam & Detik
3. Detik    --> Jam & Menit
4. Keluar
Pilihanmu: abc
Input tidak valid!
```

Gambar 4.6 Pilihan di luar Opsi dan Input tidak Valid

```
Pilihanmu: 2
Masukkan menit: -50
Nilai tidak boleh negatif!
```

Gambar 4.7 Nilai Negatif

5. Langkah-langkah GIT

Sebelum melakukan proses *Git*, kita harus memiliki akun GitHub dan membuat *repository* baru. Lalu siapkan *folder* yang akan di git dan buka Command Prompt, PowerShell, atau semacamnya untuk melakukan *git*.

5.1 Git Init

```
PS D:\praktikum-apl> git init
Initialized empty Git repository in D:/praktikum-apl/.git/
```

Gambar 5.1.1 Proses Git Init

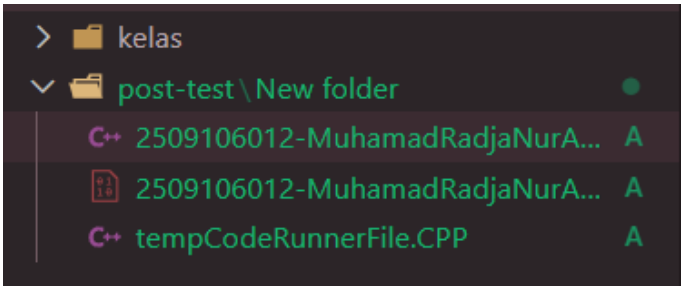
Git init bisa dilakukan terminal pada VSCode. Cukup ketik "git init" pada terminal untuk menginisialisasi/membuat *repository git* di *folder* lokal dan memantau setiap perubahan yang terjadi di dalam *folder*.

5.2 Git Add

```
PS D:\praktikum-apl> git add .
```

Gambar 5.2.1 Proses Git Add

Git add berfungsi untuk memindah semua perubahan di area kerja ke indeks, ada dua cara untuk melakukan *git add*, yang pertama dengan mengetikkan nama *file* yang ingin di add. Jika kalian ingin menambahkan banyak *file* sekaligus, ketik "." Setelah *add* maka semua *file* yang di dalam area kerja akan ditambahkan.



Gambar 5.2.2 Setelah Git Add

5.3 Git Commit

```
PS D:\praktikum-apl> git commit -m "finish pt 1"
[master (root-commit) e8f9088] finish pt 1
 3 files changed, 98 insertions(+)
 create mode 100644 post-test/New folder/2509106012-MuhamadRadjaNurAkbar-PT-1.CPP
 create mode 100644 post-test/New folder/2509106012-MuhamadRadjaNurAkbar-PT-1.exe
 create mode 100644 post-test/New folder/tempCodeRunnerFile.CPP
```

Gambar 5.3.1 Proses Commit

Git commit berfungsi untuk mengkonfirmasi setiap perubahan pada *repository* kalian dengan mengetikkan "git commit -m 'pesan yang ingin ditulis' " .

5.4 Git Remote

```
PS D:\praktikum-apl> git remote add origin https://github.com/MRadjaNurAkbarA25/
praktikum-apl.git
```

Gambar 5.4.1 Git Remote

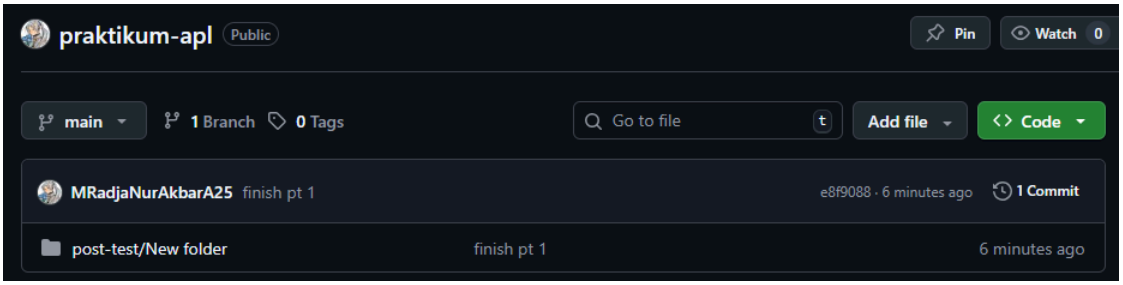
Git remote berfungsi untuk menghubungkan *repository* lokal ke *repository* yang ada di GitHub kalian, caranya dengan mengetik "git remote add origin 'link *repository* GitHub kalian' " .

5.5 Git Push

```
PS D:\praktikum-apl> git push -u origin main
Enumerating objects: 7, done.
Counting objects: 100% (7/7), done.
Delta compression using up to 12 threads
Compressing objects: 100% (5/5), done.
Writing objects: 100% (7/7), 667.93 KiB | 7.95 MiB/s, done.
Total 7 (delta 0), reused 0 (delta 0), pack-reused 0 (from 0)
```

Gambar 5.5.1 Git Push

Git push berfungsi untuk mengunggah *file* lokal tadi ke GitHub kalian dengan mengetikkan "git push -u origin main. Kembali ke *repository* di GitHub kalian, klik *refresh* dan *file* sudah terunggah.



Gambar 5.5.2 File Terunggah